EL WORKFLOW — los 26 pasos, de la idea a la estrategia superviviente

Esto lo dictó el dueño el 2026-09-23 y es la espina dorsal del proyecto. Documento vivo, sin fecha en el nombre: se actualiza, no se sustituye. Cualquier sesión que vaya a construir algo mira aquí primero en qué paso está y qué contrato tiene que cumplir.

Qué NO es: un plan de reparto entre agentes (el plan y el protocolo del 21-09 se retiraron el 26-09: sus tablas de estado contradecían el código). Esto es la **secuencia**, y manda sobre el orden que digan los otros dos.

Cuántos pasos son. Veintiséis numerados (1-26) y cuatro intermedios que no renumeran nada (10.5, 16.5, 18.5): 29 filas en la tabla. La lista de la ventana, `ui/daemon/workflow/steps.py`, tiene las mismas 29 y `tools/checks.py` («workflow table») falla si difieren en número o título. «Los 20 pasos» y «los 25 pasos» de otros textos son cifras de antes del 2026-09-29, cuando se añadió el 26; cualquier cifra distinta de 26 es un texto sin actualizar.

Estados: □ construido y medido · □ existe, sin cerrar · □ sin empezar · □ bloqueado por un dato que sólo el dueño puede dar.

La secuencia

#	paso	dónde	estado
1	**Idea en el chat**	—	—
2	**Vocabulario** — ¿existe el bloque? si no, se crea e instala	`sqx/inspect/vocabulary.py`, `sqx/blocks/install.py`	□
3	**Plantilla** — bloque fijo + hueco aleatorio, con registro de lo ya probado	`sqx/templates/`, `/sqx-strategy-template`	□
4	**Preflight** — costes, ventanas y rangos, BLOQUEANTE	`core/assets.py`; el aviso de calidad del feed lo escribe `python3 -m studies.data.feedQuality.scan` (manual `57-calidad-del-feed.md`); el spread real de Darwinex y la propuesta de coste por tramo, `python3 -m studies.data.spread.scan` (manual `59-spread-real.md`, hoy XAUUSD y USDJPY)	□ · □ aviso de calidad del feed (encargo 17, 2026-09-26): anomalías por año, año estable y episodios del proveedor; **avisa, no bloquea**. K y sesión por feed congelados en el ledger · ⚠ el Brent (109 huecos propios al año) y la plata (33) pasan del listón de 30: su sesión, deducida del feed, no recoge todas sus pausas — la del Brent la declara el dueño
5	**Creación del custom project**	`sqx/projects/builder.py`	□
6	**Configuración del build**	`assets/_build.yaml`, `sqx/projects/doctrine.py`; filtros de aceptación: `assets/_study.yaml` → `sqx/projects/rankings.py`	□ filtros escritos por el builder desde `_study.yaml` (2026-10-02), cifras PROVISIONALES hasta tu visto bueno · □ building blocks: el agente `buildingBlocksExpert` (paso 6), la taxonomía de 767 bloques (`sqx/blocks/taxonomy.yaml`) y las paletas por familia (`sqx/blocks/palettes/`, `assets/FAMILIAS.md`) existen desde el 2026-10-01; el builder aplica la paleta y la regla de 1..2 condiciones por lado (OPEN.md #93); las paletas v3 siguen en curso
7	**Retest OOS en SQX**	tarea propia, costes de `oos1`	□
8	**Análisis IS/OOS en Python** — la puerta, con una sola regla estadística, la z del mono con horas por celda: `z_mono ≥ Φ⁻¹(1 − 0,01/B)`, B = lotes cribados en oos1 en la celda (activo × marco × dirección × familia) contando éste (2,326 con B = 1; 2,576 con 2); sin dato, falla. Siguen juzgando presencia, sanidad, neto de oos1 > 0, ventaja frente al coste, degradación dentro de oos1, calidad de datos, retraso de una barra, R7 (sólo Limbo) y R6. R3, R4a, R5, R5b, R8, R9b y las escalas de la t son **columnas**, no juzgan. Tras el juicio, ≤ 4 medoides por lote	`studies/screening/gate/`, `/oos-gate` → `/curate`; criterios en `pipeline/autopilot/criteria.yaml` paso "8"; vigente: `criterios-paso-08-corrida-nula-2026-10-12`, histórico: `puerta-is-oos-paso-08-especificacion-2026-10-06`	□ regla única verificada de punta a punta el 2026-10-15 (encargo 50, frente 1: B = 2, línea 2,576, embudo 10.000 → 7.356 → 6 → 4 medoides) · se miden sobre la misma cosecha, sin juzgar: edge por coste (`studies/readings/edgeCost/report.py`), calidad del feed (`studies/data/feedQuality/report.py`), reajuste al spread real (`studies.data.spread.report`) y criba del MAE profundo (`studies.screening.maeDepth.report`)
9	**Retest crossmarkets en SQX**	`sqx/projects/crossmarket.py`	□ la tarea: la escribe `sqx/projects/crossmarket.py` desde `assets/` (mercados de `_markets.yaml`, ventana y costes de la ficha de cada mercado) y corrió de verdad el 2026-09-30 (H1, USDJPY) y el 2026-10-01 (`Test_XAUUSD_timeRangeBreakout2_M30`, sobre XAGUSD) · ⚠ costes de cada mercado = su ficha en `assets/`, provisionales donde ésta lo declare (`costs_provisional`); el dueño fijó la lista de mercados el 2026-10-02
10	**Análisis crossmarkets en Python**	`studies/transfer/crossmarket/`	□ sin skill · □ reescrito el 2026-10-07 (encargo 46): «¿la lógica viaja?», nueve hechos P1-P9 por estrategia frente a corridas al azar conjuntas (cada año se desplaza en bloque, igual en todos los mercados), en R y sobre el bruto; el código antiguo (nulos por mercado, amplitud, Fieller, Timing Alpha) se borró. Las líneas de `criteria.yaml` son PROVISIONALES y están sin calibrar · ⚠ los veredictos guardados con el estudio antiguo hay que rehacerlos
10.5	**Preparación crossTF** — generación de variantes escaladas (madres M30: M15, H1 y H4; los coeficientes por vela se dividen por el cociente de ATR medido)	`sqx/variants/scale.py`, `/crosstf`	□
11	**Retest crossTimeframes en SQX**	`/crosstf`	□
11.5	**Criba de fase 1 de crossTF en Python** — sin corte hasta el «corta» del dueño (encargo 47)	`studies/transfer/crossTF/tf/`	□

#	paso	dónde	estado
11.7	**Retest crossTF en el build en SQX** — sólo las que siguen; se salta sin tareas de fase 2 (encargo 47)	`sqx/projects/crosstfload.py`	☐
12	**Análisis crossTFs en Python** — la regla viva sigue siendo `crossTF.combined.z_new`; la lectura nueva (T0-T4, «¿pico aislado en su TF?») se emite al lado y espera la primera corrida real y el «corta» del dueño (encargo 47)	`studies/transfer/crossTF/`	☐
13	**MC Retest en SQX** — rediseño del dueño, 2026-10-07: primero el lote de vecinas por la fábrica de variantes (todos los parámetros declarados a la vez, entradas ±15 %, salidas ±30 %, hasta 500, reprobadas en WFC 1 IS y WFC 2 OOS1), luego sólo `MCR 1 Bar`, `MCR 4 MinDist` y `MCR 7 OHLC`; `MCR 2, 3, 5, 6 y 8` fuera de la etapa (su definición sigue en `assets/_build.yaml`). Orden del autopiloto: juez 12 → vecinas → SQX 13 → Python 14 → juez 14 → SQX 15	`sqx/variants/neighbourbatch.py`, `sqx/projects/mcretest.py`, `/mcretest`; etapa en `sqx/projects/stages.yaml`	☐ · el lote de vecinas, sus tiempos y el configurador sobre un proyecto sin las cinco tareas **no están probados en vivo**
14	**Análisis MC Retest en Python** — fragilidad: retención del EXCESO sobre la deriva; sobre el databank `MCR 7 OHLC`; columna, sin regla (`rules: []` en `pipeline/autopilot/criteria.yaml`, clave `"14"`): T2 y T3 se borraron (encargo 65, dueño 2026-10-12) y T1 queda como hechos `mcRetest.fragilidad.t1.*` que el 16.5 enseña. No corta ni juzga	`studies/breakage/mcRetest/` (`neighbours/`, `model/retention.py`)	☐
15	**SPPs en SQX** — `SPP IS` lee el databank `MCR 7 OHLC`; un proyecto viejo (cadena antigua) lo tiene apuntado a `MCR 8 Stress`, que ya nada rellena: `python3 -m sqx.projects.spp <SÍMBOLO> --cfx <cfx> --input "MCR 7 OHLC"` con el worker parado	`sqx/projects/spp.py`, `/spp`; catálogo en `assets/_build.yaml`	☐
16	**Análisis SPPs en Python**	`studies/breakage/spp/`	☐
16.5	**Preparación de variantes para el WFC** — la fábrica: un hipercubo latino de 1.000 a 5.000 variantes dentro de la meseta del SPP IS, las operaciones de todas ellas (`variant_trades.parquet`) y, al final, el mapa de la nube; no corta nada	`sqx/variants/make.py`, `/variants`, `studies/optimisation/cloud/`	☐ construido sin SQX · ☐ hay 16 lotes reales con `variant_trades.parquet` (2026-10-08 a 10-11)
17	**Walk Forward Correlation** — «¿elegir por el pasado sirve fuera?»: ρ de Spearman del exceso de dentro contra el de fuera frente a un nulo de volteo de signos **semanal** (una operación cuenta entera en la semana en que cierra), y la mediana de fuera del top; da **PASA / LIMBO / NO PASA** de la composición que decide (c1: build+oos1 dentro, oos2 fuera; c2 y c3 se anotan): el top gana fuera cuando su mediana neta es > 0, **sin pedir significancia** (`p_nivel ≤ 0,05` se exige además para PASA: `verdict/state.py`, `criteria.yaml`); PASA = orden no perverso y el top gana (un veto levantado, no un voto a favor), un orden perverso con el top ganando es LIMBO, un top que no gana es NO PASA. Anota también los cierres tras fin de semana, el orden con bloques de dos semanas y el descuadre con el NetProfit de SQX. El paso 20 lo traduce (PASA = sin fallo, NO PASA = fallo, LIMBO = indeciso). oos2 se abre sólo con la calibración registrada (`abre: true` desde el 2026-10-07; midió el nulo del nivel, no el criterio tal como se escribió)	`studies/optimisation/wfc/report.py`, manual `19-wfc.md`	☐ construido sin SQX · ☐ sin lote real con `variant_trades.parquet`
18	**CSCV** — «¿elegir la mejor variante del pasado sirve fuera?»: 16 bloques, 12.870 particiones, sobre el exceso semanal neto de las variantes (lee `variant_trades.parquet`); **NO PASA** si la elegida pierde fuera en la mitad de las particiones o más (`p_perdida` ≥ 0,5) o si su PBO llega al p95 de su propio nulo; **LIMBO** si `p_perdida` supera el p95 de la de su propio nulo (casi nunca salta con variantes correlacionadas) o con menos de 30 variantes; PASA = no descarta. Anota el PBO y su p, la recta de degradación (β, R^2), el recorte del Sharpe,	`studies/optimisation/cscv/report.py`, manual `25-cscv.md`	☐ construido sin SQX · ☐ sin lote real con `variant_trades.parquet`

#	paso	dónde	estado
	la dominancia estocástica, la potencia con un Sharpe plantado y la lectura de la madre. ⚠ El corte de 0,5 suspende al 29 % de los paneles correlados sin ventaja con Sharpe semanal 0,03		
18.5	**Superficies por mercado** — «¿el orden de las variantes en el mercado principal se repite en los demás?» (los de `_markets.yaml`, cuyo número depende del activo; los casi idénticos comparten un peso): ρ de Spearman media ponderada del exceso bruto contra un nulo de volteo semanal conjunto; orden positivo / neutro / perverso. **Partido en dos**: la lectura que decide (build+oos1) se corre y se juzga tras el 16.5, ANTES del bloque ciego (perverso → NO PASA; sin dato, o la prima del mejor 20 % bajo el p5 de su nulo → LIMBO); la lectura de oos2 se calcula dentro del bloque ciego y sólo puede añadir LIMBO	`studies/optimisation/marketSurfaces/report.py [--oos2]`, manual `52-superficies-mercado.md`	□ construido sin SQX · □ sólo probado con paneles sintéticos (ningún lote real tiene `variant_trades.parquet`)
19	**Walk Forward Matrix en SQX** — la tarea (30 celdas) y su lectura: sólo cuentan las operaciones que cierran en `oos2`, recobradas con los costes de oos2; **L1** (mediana de las 30 celdas del exceso neto por año contra un nulo de volteo semanal) decide: NO PASA si p− ≤ 0,05, LIMBO si no gana, PASA si gana. `modo_real` (por defecto `fijo`) añade, en `reoptimizado`, dos reglas que sólo pueden dar LIMBO. L2, celdas efectivas, deriva y el área de SQX se anotan	`sqx/projects/wfm.py`, `/wfm`; el análisis es `studies/optimisation/wfm/`, manual `34-wfm.md`	□ la tarea y la lectura (una corrida real: 10 madres del Nikkei, 4 PASA, 5 LIMBO, 1 NO PASA)
19.5	**Lectura única de oos2: el exceso neto sobre el mono con horas** — «¿la madre gana a un trader al azar con sus mismas horas en `oos2`?»: suma semanal del exceso neto contra un trader al azar con sus mismas horas, nula de 100.000 volteos de signo, una cola. **Corta sólo por signo** (z ≤ 0 fuera) y es el único corte de la cola; el nivel sólo ordena. Una mirada por celda (activo × TF × dirección × familia), contada en el ledger; con más de 3 miradas la celda queda «solo vivo» y no se lee (encargo 65, dueño 2026-10-12). `python3 -m studies.optimisation.oos2Reading.report --project <P>`		
20	**Análisis conjunto de 17, 18, 18.5 y 19** — y el SPA/StepM de cada madre contra un trader al azar con la misma huella sobre `oos2` (encargo 10 B). Lectura provisional `peor` (dueño, 2026-10-07): **manda el peor de los cuatro estados**: algún NO PASA → DESCARTAR, algún LIMBO → REVISAR, si no MANTENER; la cadena corta lo que sale DESCARTAR (2026-10-11)	`studies/closing/blindJoint/report.py`, manual `56-paso-20.md`; antes, `ledger.backfill --blind` (17-19 no se apuntan solos)	□ · □ provisional hasta que el dueño rediseñe el paso 20 · el SPA sobre `oos2` se lee para un humano; sólo un agente autónomo lo encuentra cerrado (`BlindJoint` fuera de `reserved_for`)
21	**Exposición contra el buy and hold** — qué tiempo de mercado costó lo que ganó	`studies/closing/exposure/`, `docs/manual/10-cierre.pdf` (cap. 38-exposicion)	□ · sólo describe: sin veredicto ni umbral de eficiencia (2026-10-11)
26	**Validación en MT5 con la feed de cada empresa de fondeo → pool validado** — sobre la estrategia ya dada por buena en 1-25: SQX traducido al reloj y los costes de la empresa contra el backtest de MT5 en el tramo común, comparando operaciones, P&L diario en el día del servidor, peor día y drawdown intradía con umbrales fijados antes (fila 1: el emparejamiento exigido en los dos sentidos; fila 1b: la salida también coincide; «Sin veredicto» si SQX tiene menos de `mt5verify.trades_min` operaciones, y no se anota en el pool; el primer veredicto cuenta en el pool; `exit_same_min` y `trades_min` son provisionales hasta una corrida de referencia, OPEN.md); si coincide, entra en el pool de esa empresa, lo único que lee la cartera (dueño, 2026-09-29)	encargo 34, `mt5/verify/`, ventana › MT5 BRIDGE › Verificar	□ la verificación está construida (2026-09-29): un backtest de SQX con las condiciones de cada empresa contra el del EA en MT5 en la cuenta de esa empresa, con los cinco umbrales del dueño; falta el pool, y una superviviente real

Pasos que ya no están en la cadena (sus números no se reutilizan):

- **22** — **Mapa condicional** — eliminado (2026-10-11): el estudio y su paso se borraron del código; no se renumera ningún paso

- **23** — **Estructura** — eliminado (2026-10-11): el estudio y las variantes estructurales (`make` , `keep` , `plan`) se borraron; sólo queda `sqx/structural/logic.py` , que lee la lógica de una estrategia
- **24** — **El stop loss para MT5** — eliminado (2026-10-11). La medición preregistrada dio NO PASA (`portfolio/DECISIONS.md` , entrada del 2026-10-11; dossier `medicion-stop-lejano-2026-10-11.md`): un stop lejano 2·X toca el 11,55 % de las operaciones, el límite era el 1 %. Las estrategias operan sin stop por operación y el 26 recibe el `.sqx` de la madre tal cual
- **25** — **Edge por coste, por estrategia** — fuera de la cadena (2026-10-11): ningún paso lo corre; el modo `--strategy` de `studies/readings/edgeCost/report.py` sigue como orden suelta (el paso 8 usa el modo población)

Tres lecturas adicionales que no son pasos nuevos y no renumeran nada. Dos sobre una estrategia y su lista de operaciones, gratis y sin SQX: `studies/readings/profitShape/` (manual `40-forma-del-beneficio.md`) dice de qué pocas operaciones y qué pocos meses depende el resultado, si las operaciones se agrupan —y entonces el Monte Carlo de cartera tiene que remuestrear por bloques— y si la media cambió dentro de la muestra; `studies/readings/entryQuality/` (manual `42-calidad-de-la-entrada.md`) aísla la **entrada** del resto midiendo el recorrido a favor y en contra contra entradas al azar a las mismas horas, y lo que cuesta llegar tarde. Las dos salen del PDF `TRADE_LEVEL_TESTS.pdf` del dueño. Y la tercera, sobre el lote de variantes: `studies/optimisation/cloud/` (manual `39-nube-de-parametros.md`) lee las variantes del 16.5 como lo que son —una superficie cuadrática sobre los parámetros vivos— y dice qué parámetros mandan (índices de Sobol) y si ese ranking se puede leer: sólo si el R^2 ajustado supera el de la misma superficie ajustada a la métrica barajada; si no, «sin estructura distinguible del ruido». Es gratis, no toca SQX, **lee sólo build y oos1, nunca oos2**, no tiene regla y no elige nada: sale del PDF `PARAMETER_SPACE_TESTS.pdf` del dueño, cuya sección E prohíbe expresamente sustituir la madre por el mejor clon.

Del 27 en adelante empieza la cartera, y sólo con estrategias del pool validado del paso 26. **Cómo se construye está planificado en `portfolio/PLAN.md`** (2026-09-30): primero el universo común (M0-M2), luego el camino del fondeo (F1-F5: reglas de cada empresa, cartera elegida por la probabilidad de aprobar sobre el flotante, veredicto en `oos1` + `oos2` , caja del encargo 33), luego la cuenta real (M3-M8) y el OOS en papel sobre barras nuevas (M9). Hasta el paso 26, se desarrolla con supervivientes de cualquier paso archivadas como desarrollo, en USDJPY y XAUUSD. **Primero las estrategias individuales.** Todo lo que usa `oos2` para desarrollar una estrategia ocurre antes, en 1-25; el paso 26 no elige por rendimiento, sólo comprueba que SQX reproduce la cuenta de la empresa (dueño, 2026-09-29).

El paso 21 — la dicotomía rendimiento/exposición

Encargo del dueño, 2026-09-24, y cierra la secuencia individual:

> *«Una estrategia que saque 5 %, pero solo se exponga al mercado 1 hora a la semana, me parecería mejor que sacar un 10 % con el buy and hold.»*

Los pasos 1 a 20 preguntan si la ventaja es **real**. El 21 pregunta qué **forma** tiene: el mismo retorno conseguido con el 4 % de ocupación no es el mismo objeto que conseguido estando dentro siempre, porque las horas fuera no tienen gap, ni noticia, ni drawdown, ni capital inmovilizado.

Sólo describe (2026-10-11): sin veredicto ni umbral de eficiencia. Mide tres cosas y no las confunde: lo que ganó **en total** contra un buy and hold **al mismo riesgo**, lo que gana **por hora expuesta**, y cuánto del movimiento del mercado ocurrió mientras tenía posición — que es lo que separa una ventaja propia de estar presente en los tramos buenos.

La alfa y la beta están aparcadas ahí, a propósito. Decisión del dueño del mismo día: primero se cierra la secuencia individual. Si algún día se construye la regresión, va en `studies/closing/exposure/` , que ya tiene montada su antesala.

El paso 24 — eliminado (2026-10-11)

La cadena construye **sin stop a propósito**: un stop es un parámetro más y cada parámetro es sitio para el sobreajuste. MT5 lo exigía, y el paso 24 intentó dárselo con un stop lejano `SL = X·ATR` leído del MAE. La medición preregistrada del 2026-10-11 dio **NO PASA**: ese stop toca el 11,55 % de las operaciones (el límite era el 1 %), es decir, ya no es un colchón sino un filtro. Decisión del dueño: el paso, su estudio (`atrCalculator`) y su injerto en SQX se borraron. Las estrategias operan sin stop por operación y el 26 recibe el `.sqx` de la madre. Registro: `portfolio/DECISIONS.md` (2026-10-11), `docs/AgentPDFs/medicion-stop-lejano-2026-10-11.md` y OPEN.md #113 (lo que queda: el dimensionado de la cartera aún supone un stop por operación).

El Monte Carlo de bootstrap está FUERA, a propósito

Decisión del dueño, 2026-09-23, y el razonamiento merece quedar escrito porque es correcto:

> Todos esos tests que hay antes están dedicados a intentar descartar que la estrategia haya tenido suerte en el backtest. Mientras que el Monte Carlo sólo resuelve los trades de un backtest estático, no dice nada del edge.

Remuestrear los trades realizados mide la dispersión de **una muestra ya seleccionada**. Si la estrategia llegó ahí por haber sobrevivido a una búsqueda, el bootstrap hereda esa selección entera y no puede verla: reordenar trades afortunados da secuencias afortunadas.

El módulo `portfolio/common/monteCarlo/` se mantiene: en la cartera responde otra pregunta — dimensionamiento y riesgo de ruina dada una distribución de trades— y ahí sí es la herramienta.

Qué segmento toca cada paso

Sale de `assets/_policy.yaml` y no se negocia por paso:

- `build` — sólo el paso 6. Es la única muestra que el generador ve. El mapa condicional (22, eliminado) también lo leía, sólo para congelar sus cortes de tercil — clasificaba la muestra de `run.sample` (por defecto `oos1`), nunca la construye contra ella.
- `oos1` — del 7 al 16 (junto con `build`).
- `oos2` — lo leen 17, 18, 18.5, 19 y 19.5 (WFC, CSCV, superficies por mercado, WFM y la lectura única). Cada lectura deja su fila en el ledger.

Un humano puede mirar cualquier tramo cuando quiera (dueño, 2026-09-28). La regla de «no quemar el oos2» —puerta de un solo sentido, `reserved_for` de `_policy.yaml`, paso 20 ciego— queda **sólo para un agente autónomo que tome solo las decisiones de desarrollo**, y aun ahí está por discutir. En código: `core.assetdata.enforced()` es falso salvo con `ALGO_AUTONOMOUS=1`, y sin él ni `ledger.gate`, ni la ventana, ni los configuradores de SQX rechazan ni sellan nada. El ledger sigue contando cada mirada: es información para el Sharpe desinflado, no un candado.

El contrato que une los pasos pares

Todo paso de Python que juzgue estrategias emite **un CSV con `strategy` y `verdict`**, y `/curate` lo aplica sobre el databank para que el siguiente paso de SQX sólo vea a los supervivientes. Da igual qué haya juzgado: el mecanismo no cambia. `identity` es opcional pero recomendada — el nombre no identifica, SQX renombra en colisión.

⚠ La identidad es el SHA-256 del `strategy_Portfolio.xml` **normalizado**. Un retest reescribe `makeExternal` en cada variable: con el hash crudo, el emparejamiento build↔retest casa 0 de 115.

Decisión del dueño — 2026-09-29: del 16 al final, todas las supervivientes por todos los pasos

> «Se corren todos los steps con todo lo que sobreviva a los SPP y a su análisis. […] En un workflow profesional, que corran los 4.»

- **Workflow profesional**: el embudo corta hasta el paso 16 (8, 10, 12, 14 y 16 juzgan y su veredicto se aplica antes de la siguiente tarea de SQX). Lo que sobrevive al 16 pasa **entero** por 16.5, 17, 18, 18.5 y 19 —las cuatro piezas, sin filtrar una con otra— y el 20 las lee juntas; 21 (y el 26, cuando se piden sus cuatro parámetros) se corre sobre lo que salga del 20; 22-25 ya no están en la cadena.
- **No** se filtra con la WFM (ni con ninguna de las cuatro) antes de correr las demás.
- **Excepción**: con otro fin que no sea sacar una estrategia rentable —probar o mejorar el código—, se puede correr un solo paso suelto; la ventana lo permite con el ▶ de cada tarjeta.

Decisiones del dueño sobre la secuencia — 2026-09-23

El paso 20 va ciego, y por eso el orden importa — sólo para un agente autónomo

> **Derogado para humanos el 2026-09-28.** Lo que sigue es la regla original; hoy sólo ata a un agente autónomo (`ALGO_AUTONOMOUS=1`), y si debe seguir atándolo está por discutir. Con la ventana o una sesión dirigida por el dueño, 17, 18 y 19 se leen cuando se quiera.

Los pasos 17, 18 y 19 **no se miran hasta que los tres estén hechos**. La razón no es estética:

> Una vez miramos una ventana de OOS la quemamos, y esas pruebas se hacen con los datos que ya habíamos visto **y además la última bala del `oos2`**.

Del 7 al 16 se mira `oos1` una y otra vez, así que para cuando se llega al 17 ese tramo está gastado. `oos2` es lo único virgen que queda, y sólo se dispara una vez. Si se leen los resultados de 17 y 18 antes de correr el 19, la decisión de si correrlo —y con qué parámetros— ya está contaminada por lo que se vio, y la última bala se gasta en un test elegido a posteriori.

Esto ha de estar forzado por el ledger, no por la buena voluntad de quien lo corra.

□ **Construido el 2026-09-24**, y desde el 2026-09-28 activo sólo con `ALGO_AUTONOMOUS=1`. `ledger/gate.py` se niega: `allow(paso, segmento, activo)` lanza si un paso mira un tramo que `_policy.yaml` reserva para otro, y `allow_read` lanza si se piden los resultados de 17, 18 o 19 sin que los tres estén registrados. Manual: `docs/manual/03-datos-costes-y-registro.pdf` (cap. 43-ledger).

Qué significa «los tres a la vez» — aclarado 2026-09-24

No significa correrlos en paralelo. **Corren secuencialmente, y así se quiere**: el 17, el 18 y el 19 uno detrás de otro. Lo que tiene que ser simultáneo es **la lectura**:

> Cuando digo que quiero ver el WFC + WFM + CSCV a la vez, me refiero al mismo tiempo y una vez ya sepamos que el programa funciona bien. La cosa es que, cuando en un futuro hagamos una UI, los resultados de las 3 pruebas se mostrarán a la vez, o al menos no ver uno mientras se está simulando la siguiente prueba.

~~Consecuencia para quien programe la ventana de `ui/`: los tres resultados se retienen hasta que los tres existen.~~ **Derogado el 2026-09-28:** la ventana la maneja un humano, así que no retiene nada; enseña cada resultado en cuanto existe.

Un manifiesto global de la cadena — acordado

Un solo registro por estudio que diga, paso a paso, cuántas estrategias entran y cuántas salen. Sin él, al llegar al 17 nadie sabe si quedan tres supervivientes de diez mil o de cincuenta — y esa diferencia **es** el resultado, no un detalle de contabilidad.

☐ Antes de correr el workflow entero por primera vez: pasar una población NULA

Recordatorio para el dueño, pedido por él el 2026-09-23.

La cadena son seis filtros en serie sobre la misma población, y cada uno ajusta su umbral mirando lo que mató el anterior. Eso es selección múltiple: el superviviente del paso 20 ha pasado por una búsqueda tan intensa como la del builder, sólo que esa búsqueda **la hacemos nosotros**.

Habrá métricas y requisitos de rendimiento cuantificables, y eso es necesario. No es suficiente: un umbral cuantificado sigue sin decir cuántas estrategias sin ningún edge lo cruzarían por azar.

Lo que sí lo dice: **meter por la cadena entera una población de estrategias de entrada aleatoria** y contar cuántas llegan al paso 20. Si de diez mil monos llegan 4 y de diez mil tuyas llegan 5, ya sabes lo que vale ese 5. La máquina del mono ya existe (`studies/screening/gate/monkey.py`) y hoy sólo se usa en el paso 8; llevarla de punta a punta es el control que hace interpretable todo lo demás.

Hacerlo **antes** del primer pase completo, no después: después ya se sabe qué sobrevivió, y el número de monos se convierte en una cifra que se compara con un resultado conocido.

Lo que bloquea hoy

	qué falta, y de quién depende
☐	**`assets/_study.yaml`** — existe desde el 2026-10-02 y el builder lo escribe en cada proyecto nuevo (operaciones por año según timeframe, profit factor ≥ 1,20, beneficio > 0, todo sobre el IS, `sampleType` 10; los tres filtros heredados del donante, fuera). Ya no bloquea construir; queda **tu visto bueno a las cifras**, que son propuesta de agente: `docs/AgentPDFs/criterios-implementados-2026-10-12.md` y el A/B de suelos dentro del build en `AlgoData/scratch/calib_final/ab_build.md` (borrado el 2026-10-04). Del dueño
☐	**los building blocks** del paso 6 — existen el agente `buildingBlocksExpert` , la taxonomía y las paletas por familia (ver la fila 6 de la tabla); las paletas v3 las está terminando otra sesión, y siguen abiertas la pregunta de los bloques de reloj (`SessionHigh/Low` , `HighestInRange` , `CBlock_TimeBreakout*`) y la de la regla de salida en OPEN.md #93
☐	**el paso 20 construido, sin decidir**: qué es «pasar» (unanimidad / sin_fallo / sólo el StepM, y K = supervivientes o entrantes). Leer `oos2` ya no lo bloquea para un humano (2026-09-28). Del dueño — `OPEN.md` §50
☐	**el motor de carteras (paso 27 en adelante), construido y sin ningún resultado real** — `portfolio/PLAN.md` §9: M0-M9 y F1-F5 construidos el 2026-09-30 y el 2026-10-05, con pruebas de respuesta conocida, y corridos sólo sobre el pool relajado `Test_nikkei_ref` en una raíz de prueba (ni un pool real, ni `oos2` real leído, la incubación no corre). Respondidas el 2026-10-05: Q10, Q11, Q16, R1-R8, recompra y recortes. Siguen abiertas, del dueño: la fracción del tope de riesgo abierto, `verdict.pass_percentile` , `stageb.t1_gap` , `wf.confidence` , `incubation.*` , las nueve entradas del banco de fondeo y las lecturas que el código tomó (PLAN §12). Hallazgos sin arreglar: `OPEN.md` #104. Del dueño — `OPEN.md` #87, #85
☐	**calidad del feed** (encargo 17, construido 2026-09-26) — queda del dueño: la **sesión del Brent** con sus pausas (109 huecos propios al año con la deducida); la inyección la dio por buena el 2026-09-26. Tres detalles de diseño abiertos en `studies/data/feedQuality/POSSIBLE_IMPROVEMENTS.md`